Mahatma Education Society's

Pillai College of Arts, Commerce & Science

(Autonomous)

Affiliated to University of Mumbai

NAAC Accredited 'A' grade (3 cycles)

Best College Award by University of Mumbai

ISO 9001:2015 Certified

CERTIFICATE

This is to certify that Mr. ***Rohit Malviya*** *of* ***M.Sc. D.A. Part-II Semester IV*** *has completed the practical work in the Subject of* ***Natural Language Processing*** *during the academic year 2024-25 under the guidance of Prof.* ***Shivapradeep Muthupandi*** *being the partial requirement for the fulfillment of the curriculum of* ***Degree of Master of Science in Data Analytics, University of Mumbai****.*

Place:

Date:

Name & Signature of faculty *Name & Signature of external*

INDEX

Sr. no.	Details	Date	Signature
1.	Basics of NLP(Natural Language Processing) ● Text-to-Speech ● Porter Stemmer ● Lemmatization		
2.	Regular Expressions		
3.	● Write a program to generate N-gram(unigram,bigram,trigram) ● Write a program to find unicode point for the following: a. English letter: b. Hindi letter: c. Emoji: ● Generate a fake corpus using faker library and extract important word from the corpus using TF-IDF.		
4.	Write a program to tokenize the sentences and words, identify and remove stop words.		
5.	Generate a word cloud for the above generated fake corpus		
6.	Write a program to identify Part of Speech of given english text.		
7.	Write a python program to perform Context Free Grammar(CFG) and print parse tree.		
8.	Write a python program to perform Probabilistic Context-Free Grammar (PCFG) and print Probability value and parsing tree.		
9.	Write a function for morphological parser using spacy library and extract text, corresponding POS.		
10.	Write a function for morphological parser using NLTK library and extract text, corresponding POS and morphological tag		

Practical 1

Aim: Basics of NLP(Natural Language Processing)

- Text-to-Speech
- Porter Stemmer
- Lemmatization

```
!pip install gTTS
from gtts import gTTS
from IPython.display import Audio

# Enter the text you want to convert
text = "Hello there! Its japanese"

# Create the speech object for english
tts = gTTS(text=text, lang='en')
# Save to file
tts.save("speech.mp3")

# Play the audio
Audio("speech.mp3")
```

PorterStemmer V/S Lemmatization

PorterStemmer

```
import nltk
from nltk.stem import PorterStemmer
stemmer = PorterStemmer()
word = "crying"
stemmed_word = stemmer.stem(word)
print(f"Original word: {word}, Stemmed word: {stemmed_word}")
```

```
Original word: crying, Stemmed word: cri
```

LEMMATIZATION

```
nltk.download('punkt_tab')
nltk.download('wordnet')
nltk.download('omw-1.4')
nltk.download('averaged_perceptron_tagger_eng')
```

Show hidden output

```
import nltk
from nltk.stem import WordNetLemmatizer

lemmatizer = WordNetLemmatizer()

word1 = "replacement"
lemma1 = lemmatizer.lemmatize(word1, pos='v') # Specify verb POS for better accuracy
print(f"Lemmatized '{word1}': {lemma1}")

word2 = "crying"
lemma2 = lemmatizer.lemmatize(word2) # Default to noun POS
print(f"Lemmatized '{word2}': {lemma2}")
```

```
Lemmatized 'replacement': replacement
Lemmatized 'crying': cry
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
```

Practical 2

Aim: Regular Expression

```python
import re

print("\n" + "="*50)
print("1. Extract all email addresses")
print("="*50)
text1 = """Thank you for contacting us. For product support, write to support@shopnow.com or
backup_support@shopnow.in. You can also reach the sales team at sales.team2025@gmail.com."""

emails = re.findall(r"[\w\.-]+@[\w\.-]+\.\w+", text1)
print("Emails:", emails)
```

```
==================================================
1. Extract all email addresses
==================================================
Emails: ['support@shopnow.com', 'backup_support@shopnow.in', 'sales.team2025@gmail.com']
```

```python
print("2. Remove all special characters except letters, numbers, spaces")
print("="*50)
text2 = "Welcome!!! This---text $$$contains%%% a lot of ###special*** characters@@@ and 1234 numbers…"
cleaned_text2 = re.sub(r"[^A-Za-z0-9 ]", "", text2)
print("Cleaned Text:", cleaned_text2)
```

```
2. Remove all special characters except letters, numbers, spaces
==================================================
Cleaned Text: Welcome Thistext contains a lot of special characters and 1234 numbers
```

```python
print("3. Check if strings contain only alphabets")
print("="*50)
samples = ["MachineLearning", "Data123", "hello_world"]
for s in samples:
    if re.fullmatch(r"[A-Za-z]+", s):
        print(f"{s} → Valid")
    else:
        print(f"{s} → Invalid")
```

```
3. Check if strings contain only alphabets
==================================================
MachineLearning → Valid
Data123 → Invalid
hello_world → Invalid
```

```python
print("4. Extract all hashtags")
print("="*50)
text4 = "Had a great time learning NLP today! #NLP #AI #DeepLearning #PillaiCollege #NLP2025"
hashtags = re.findall(r"#\w+", text4)
print("Hashtags:", hashtags)
```

```
4. Extract all hashtags
==================================================
Hashtags: ['#NLP', '#AI', '#DeepLearning', '#PillaiCollege', '#NLP2025']
```

```python
print("5. Extract words starting with capital letters")
print("="*50)
text5 = "Florence teaches at Pillai College in New Panvel and is interested in Natural Language Processing."
capital_words = re.findall(r"\b[A-Z][a-zA-Z]*\b", text5)
print("Capitalized Words:", capital_words)
```

```
5. Extract words starting with capital letters
==================================================
Capitalized Words: ['Florence', 'Pillai', 'College', 'New', 'Panvel', 'Natural', 'Language', 'Processing']
```

```python
print("6. Validate Indian phone numbers (+91-XXXXXXXXXX)")
print("="*50)
numbers = ["+91-9876543210", "9876543210"]
for n in numbers:
    if re.fullmatch(r"\+91-\d{10}", n):
        print(f"{n} → Valid")
    else:
        print(f"{n} → Invalid")
```

```
6. Validate Indian phone numbers (+91-XXXXXXXXXX)
==================================================
+91-9876543210 → Valid
9876543210 → Invalid
```

```python
print("7. Extract dates (DD/MM/YYYY)")
print("="*50)
text7 = "The workshop will start on 05/12/2025. The next batch is planned on 20/01/2026 and results will be declared by 31/01/2026."
dates = re.findall(r"\b\d{2}\/\d{2}\/\d{4}\b", text7)
print("Dates:", dates)
```

```
7. Extract dates (DD/MM/YYYY)
==================================================
Dates: ['05/12/2025', '20/01/2026', '31/01/2026']
```

```python
print("8. Remove stopwords (the, is, in, at, of, a, to)")
print("="*50)
text8 = "The cat is sitting at the edge of the table in a relaxed mood."
cleaned_text8 = re.sub(r"\b(the|is|in|at|of|a|to)\b", "", text8, flags=re.IGNORECASE)
cleaned_text8 = re.sub(r"\s+", " ", cleaned_text8).strip()  # remove extra spaces
print("Cleaned Text:", cleaned_text8)
```

```
8. Remove stopwords (the, is, in, at, of, a, to)
==================================================
Cleaned Text: cat sitting edge table relaxed mood.
```

```python
print("9. Extract sentences ending with a question mark")
print("="*50)
text9 = "How are you today? I hope you are enjoying the NLP class. Do you want more examples on regular expressions?"
questions = re.findall(r"[^?]*\?", text9)
print("Questions:", questions)
```

```
9. Extract sentences ending with a question mark
==================================================
Questions: ['How are you today?', ' I hope you are enjoying the NLP class. Do you want more examples on regular expressions?']
```

```python
print("10. Extract all numbers (integers & decimals)")
print("="*50)
text10 = "I bought this course for 499.50 rupees and I would rate it 4.8 out of 5. Earlier it was priced at 750."
numbers = re.findall(r"\d+(?:\.\d+)?", text10)
print("Numbers:", numbers)
```

```
10. Extract all numbers (integers & decimals)
==================================================
Numbers: ['499.50', '4.8', '5', '750']
```

+ Code + Text

```python
print("11. Validate password strength")
print("="*50)
passwords = ["Abc@1234", "weakpass", "Hello123", "Strong@Pass2025"]
pattern = r"^(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{8,}$"

for pwd in passwords:
    if re.fullmatch(pattern, pwd):
        print(f"{pwd} → Strong")
    else:
        print(f"{pwd} → Weak")
```

```
11. Validate password strength
==================================================
Abc@1234 → Strong
weakpass → Weak
Hello123 → Weak
Strong@Pass2025 → Strong
```

```python
print("12. Identify repeated words")
print("="*50)
text12 = "This is is a simple simple test sentence to to check repeated words in in a line."
repeated = re.findall(r"\b(\w+)\s+\1\b", text12)
print("Repeated Words:", repeated)
```

```
12. Identify repeated words
==================================================
Repeated Words: ['is', 'simple', 'to', 'in']
```

Practical 3

Aim:

- Write a program to generate N-gram(unigram,bigram,trigram)
- Write a program to find unicode point for the following:
    a. English letter:
    b. Hindi letter:
    c. Emoji:
- Generate a fake corpus using faker library and extract important word from the corpus using TF-IDF.

```
!pip install faker

# ----------------------------------------------
# 1. N-GRAM GENERATION (Unigram, Bigram, Trigram)
# ----------------------------------------------

from nltk.util import ngrams
from nltk.tokenize import word_tokenize
import nltk
nltk.download('punkt')
nltk.download('punkt_tab')

text = "Natural language processing makes computers understand
human language."
tokens = word_tokenize(text)
# Unigram
unigrams = list(ngrams(tokens, 1))
# Bigram
bigrams = list(ngrams(tokens, 2))
# Trigram
trigrams = list(ngrams(tokens, 3))
print("Unigrams:", unigrams)
print("Bigrams:", bigrams)
print("Trigrams:", trigrams)


# -------------------------------------------------------
# 2. UNICODE POINTS FOR ENGLISH, HINDI AND EMOJI LETTERS
# -------------------------------------------------------

english_letters = ['A', 'g', 'M', 'z']
hindi_letters = ['क', 'न', 'भ', 'ष']
```

```python
emojis = ['😀', '🔥', '❤', '🤖']

print("\nUnicode Points:")
print("English:")
for ch in english_letters:
    print(f"{ch} → {ord(ch)}")

print("\nHindi:")
for ch in hindi_letters:
    print(f"{ch} → {ord(ch)}")

print("\nEmojis:")
for ch in emojis:
    print(f"{ch} → {ord(ch)}")


# ---------------------------------------------------------------
# 3. GENERATE FAKE CORPUS USING FAKER & EXTRACT IMPORTANT WORDS
# ---------------------------------------------------------------

from faker import Faker
from sklearn.feature_extraction.text import TfidfVectorizer

fake = Faker()

# Generate 10 fake sentences
corpus = [fake.text() for _ in range(10)]

print("\nFake Corpus:")
for line in corpus:
    print("- ", line)

# TF-IDF
vectorizer = TfidfVectorizer(stop_words='english')
X = vectorizer.fit_transform(corpus)

# Extracting top important words
feature_names = vectorizer.get_feature_names_out()
tfidf_scores = X.sum(axis=0).A1
with score
word_score_pairs = list(zip(feature_names, tfidf_scores))

# Sort by highest score
```

```python
important_words = sorted(word_score_pairs, key=lambda x: x[1],
reverse=True)

print("\nTop 10 Important Words (TF-IDF):")
for word, score in important_words[:10]:
    print(f"{word}: {score:.4f}")

# Pair word
```

```
Collecting faker
  Downloading faker-38.2.0-py3-none-any.whl.metadata (16 kB)
Requirement already satisfied: tzdata in /usr/local/lib/python3.12/dist-packages (from faker) (2025.2)
Downloading faker-38.2.0-py3-none-any.whl (2.0 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.0/2.0 MB 27.2 MB/s eta 0:00:00
Installing collected packages: faker
Successfully installed faker-38.2.0
Unigrams: [('Natural',), ('language',), ('processing',), ('makes',), ('computers',), ('understand',), ('human',), ('language',), ('.',)]
Bigrams: [('Natural', 'language'), ('language', 'processing'), ('processing', 'makes'), ('makes', 'computers'), ('computers', 'understand'), ('under
Trigrams: [('Natural', 'language', 'processing'), ('language', 'processing', 'makes'), ('processing', 'makes', 'computers'), ('makes', 'computers',

Unicode Points:
English:
A → 65
g → 103
M → 77
z → 122

Hindi:
क → 2325
न → 2344
भ → 2349
ष → 2359

Emojis:
😜 → 128512
🔥 → 128293
❤ → 10084
🤯 → 129302
```

```
Fake Corpus:
-  Over condition both change child. Pm available maybe later green provide develop. Find stand ten office front else hair.
Win reason become.
Meet write instead character. Itself guy ten hour yard.
-  Which relationship nature among themselves end.
Fund place look many ability present less. Network soon draw so. Key successful industry down.
-  Activity easy can Mr population begin. Left indeed travel manage right cold.
Fish behavior company since goal one. Boy speak dark laugh force important course.
-  Above laugh drug. Say real stand student.
Including character find respond. Loss operation old stay glass our drive.
-  Outside police right western. Beautiful PM within heart recent member. Social stand control phone. Thought we particular toward perform.
-  Share cup respond nice may everybody its its. Hand third trip manage.
Fire girl may. Home standard energy pretty.
Draw near rule anything sea man. Stand information identify suffer.
-  Sense drive bill mouth site lawyer recent. Doctor may threat several nature hard strong. Similar street social every alone response. Rest
-  Fear whether either. Discussion statement student arrive product less. Who run admit. Who behavior strong star keep tree.
Discussion employee today. Those city majority.
-  Successful rise stay suddenly. Identify sit money lot box. Leader west fall executive major nearly what.
-  Resource already page produce party. Half reveal share read rule according. Guy fine food hit executive.
Successful find Congress window myself business book. Body response view role.

Top 10 Important Words (TF-IDF):
stand: 0.6541
successful: 0.5650
discussion: 0.4692
stay: 0.4674
drive: 0.4500
nature: 0.4479
student: 0.4373
recent: 0.4367
social: 0.4367
laugh: 0.4298
```

Practical 4

Aim: Write a program to tokenize the sentences and words, identify and remove stop words.

```
import nltk
nltk.download('punkt_tab')

from nltk.tokenize import word_tokenize
text = "Hello there! I'm learning NLP, and it's quite interesting.Let's explore tokenization."

tokens_nltk = word_tokenize(text)
print(tokens_nltk)
```

```
['Hello', 'there', '!', 'I', "'m", 'learning', 'NLP', ',', 'and', 'it', "'s", 'quite', 'interesting',
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
```

```
from nltk.tokenize import sent_tokenize

sentences = sent_tokenize(text)
print(sentences)
```

```
['Hello there!', "I'm learning NLP, and it's quite interesting.", "Let's explore tokenization."]
```

```
corpus = [
    "NLP is an exciting field of artificial intelligence.",
    "We are learning how to remove stop words from text.",
    "Tokenization and preprocessing are important steps.",
    "Deep learning models help improve NLP tasks."
]

print(corpus)
```

```
['NLP is an exciting field of artificial intelligence.', 'We are learning how
```

```
import nltk
nltk.download('punkt')
nltk.download('stopwords')

from nltk.corpus import stopwords
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
```

```python
stop_words = set(stopwords.words("english"))

cleaned_corpus_nltk = []

for doc in corpus:
    tokens = word_tokenize(doc.lower())
    filtered = [word for word in tokens if word.isalnum() and word not in stop_words]
    cleaned_corpus_nltk.append(" ".join(filtered))

print("Cleaned Corpus (NLTK):")
for c in cleaned_corpus_nltk:
    print(c)
```

```
Cleaned Corpus (NLTK):
nlp exciting field artificial intelligence
learning remove stop words text
tokenization preprocessing important steps
deep learning models help improve nlp tasks
```

Practical 5

Aim: Generate a word cloud for the above generated fake corpus.

[1]
19s

```
!pip install faker wordcloud matplotlib
```

Show hidden output

[2]
0s

```
from faker import Faker

# Initialize Faker
fake = Faker()

# Generate fake corpus
corpus = []

for i in range(500):
    corpus.append(fake.sentence())

# Combine sentences into one text
fake_corpus = " ".join(corpus)

print(fake_corpus[:500])  # preview first 500 characters
```

```
Eat truth project. Seek technology executive carry trial. Drug create authority probably a
```

[3]
2s

```
from wordcloud import WordCloud
import matplotlib.pyplot as plt

# Generate word cloud
wordcloud = WordCloud(
    width=800,
    height=400,
    background_color='white'
).generate(fake_corpus)

# Display word cloud
plt.figure(figsize=(10,5))
plt.imshow(wordcloud)
plt.axis("off")
plt.show()
```

sometimes
behavior
continue
night
owner
trade
impact
small
environment
community
drug
artist
building
someone
past
including
product
teacher
service
decision
Media
serious
nothing
simply
rise
Democrat
mention
kitchen
article
total
movement
factor
goal
single
growth
white
run
modern
short
television
individual
performance
face
join
heart
leg
policy
available
discussion
parent
partner
whether
especially
inside
never
outside
name
hard
common
thousand
significant
catch
discuss
Western
important
position
call
scene
bed
break
decade
safe
grow
result
movie
fact
play
kid
hand
drop
agreement
Congress
prepare
six
Future
action
move
director
system
need
stand
next
general
industry
cost
assume
health
fly
form
Involve
course
practice
institution
civil
billion
generation
foreign
lot
high
month
color
wonder
pattern
man
edge
almost
history
response
maintain
fear
government
ahead
various
Account
majority
state
model
east
buy
job
ball
Simple
Note
source
window
value
clear
away
develop
pay
turn
entire
drive
toward
prove
least
father
bill
left
may
defense
camera
give
beautiful
two
read
professor
indeed
feel
must
plant
wait
air
another
project
Wife
sign
forward
present
end
energy
hotel
enough
place
Eat
strategy
plan
main
former
wide
certainly
benefit
professional
care
respond
analysis
senior
site
opportunity
tonight
day
political
national
laugh
try
street
west
space

Practical 6

Aim: Write a program to identify Part of Speech of given english text.

```python
# Part of Speech Tagging Program

import nltk

# Download required resources
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('punkt_tab')
# Input text
text = input("Enter an English sentence: ")

# Tokenization
words = nltk.word_tokenize(text)

# POS Tagging
pos_tags = nltk.pos_tag(words)

# Display result
print("\nWord\t\tPart of Speech")
print("--------------------------------")
for word, tag in pos_tags:
    print(f"{word}\t\t{tag}")
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     /root/nltk_data...
[nltk_data]   Unzipping taggers/averaged_perceptron_tagger_eng.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
Enter an English sentence: Enter an English sentence: NLP is very interesting and useful

Word            Part of Speech
--------------------------------
Enter           NNP
an              DT
English         JJ
sentence                NN
:               :
NLP             NN
is              VBZ
very            RB
interesting             JJ
and             CC
useful          JJ
```

Practical 7

Aim: Write a python program to perform Context Free Grammar(CFG) and print parse tree.

```
pip install nltk
```

```
Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-packages (3.9.1)
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages (from nltk) (8.3.1)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from nltk) (1.5.3)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.12/dist-packages (from nltk) (2025.11.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from nltk) (4.67.1)
```

```python
import nltk
from nltk import CFG
from nltk.parse import ChartParser
# Step 1: Define the Context Free Grammar
grammar = CFG.fromstring("""
S -> NP VP
NP -> Det N
VP -> V NP
Det -> 'the' | 'a'
N -> 'dog' | 'cat'
V -> 'chased' | 'saw'
""")
# Step 2: Create a parser
parser = ChartParser(grammar)
# Step 3: Input sentence
sentence = "the dog chased the cat"
words = sentence.split()
# Step 4: Parse the sentence and print parse tree
for tree in parser.parse(words):
    print(tree)
    tree.pretty_print()
```

```
(S (NP (Det the) (N dog)) (VP (V chased) (NP (Det the) (N cat))))
                S
        ________|_____
       |              VP
       |          _____|___
       NP        |         NP
   ___|___       |      ___|___
 Det      N     V    Det      N
  |       |     |     |       |
 the     dog  chased the     cat
```

Practical 8

Aim: Write a python program to perform Probabilistic Context-Free Grammar (PCFG) and print Probability value and parsing tree.

```python
import nltk
from nltk import PCFG
from nltk.parse import ViterbiParser
# Step 1: Define the Probabilistic Context-Free Grammar
grammar = PCFG.fromstring("""
S -> NP VP [1.0]

NP -> Det N [0.6]
NP -> 'John' [0.4]

VP -> V NP [0.7]
VP -> V [0.3]

Det -> 'the' [0.8]
Det -> 'a' [0.2]

N -> 'dog' [0.5]
N -> 'cat' [0.5]

V -> 'chased' [0.6]
V -> 'saw' [0.4]
""")
```

```python
V -> 'chased' [0.6]
V -> 'saw' [0.4]
""")
# Step 2: Create Viterbi Parser
parser = ViterbiParser(grammar)
# Step 3: Input sentence
sentence = "John chased the dog"
words = sentence.split()
# Step 4: Parse sentence and print parse tree with probability
for tree in parser.parse(words):
    print(tree)
    print("Probability:", tree.prob())
    tree.pretty_print()
```

```
(S (NP John) (VP (V chased) (NP (Det the) (N dog)))) (p=0.04032)
Probability: 0.040319999999999995
              S
   ___________|___
  |               VP
  |      _________|___
  |     |             NP
  |     |          ___|___
  NP    V         Det      N
  |     |          |       |
 John chased      the     dog
```

Practical 9

Aim: Write a function for morphological parser using spacy library and extract text, corresponding POS.

```
!pip install spacy
```

Show hidden output

```
!python -m spacy download en_core_web_sm
```

Show hidden output

```
import spacy

# Load English language model
nlp = spacy.load("en_core_web_sm")

def morphological_parser(text):
    """
    This function performs morphological parsing
    and returns token text with corresponding POS.
    """
    doc = nlp(text)

    result = []
    for token in doc:
        result.append((token.text, token.pos_))

    return result
```

```
text = "The children are playing in the garden"
morphological_parser(text)
```

```
[('The', 'DET'),
 ('children', 'NOUN'),
 ('are', 'AUX'),
 ('playing', 'VERB'),
 ('in', 'ADP'),
 ('the', 'DET'),
 ('garden', 'NOUN')]
```

Practical 10

Aim: Write a function for morphological parser using NLTK library and extract text, corresponding POS and morphological tag.

```python
import nltk

# Download required resources (run once)
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')

def morphological_parser(text):
    """
    Performs morphological parsing using NLTK
    and extracts token, POS, and morphological tag
    """
    # Tokenization
    tokens = nltk.word_tokenize(text)

    # POS Tagging
    pos_tags = nltk.pos_tag(tokens)

    print("Word\t\tPOS Tag\t\tMorphological Info")
    print("-" * 50)

    for word, pos in pos_tags:
        morph = get_morphology(pos)
        print(f"{word}\t\t{pos}\t\t{morph}")
```

```python
def get_morphology(pos):
    """
    Maps POS tags to morphological information
    """
    if pos == "NNS":
        return "Plural Noun"
    elif pos == "NN":
        return "Singular Noun"
    elif pos == "VBD":
        return "Verb (Past Tense)"
    elif pos == "VBG":
        return "Verb (Gerund/Present Participle)"
    elif pos == "VBN":
        return "Verb (Past Participle)"
    elif pos == "VBZ":
        return "Verb (3rd Person Singular Present)"
    elif pos == "JJ":
        return "Adjective"
    elif pos == "RB":
        return "Adverb"
    else:
        return "Other"
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     /root/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]       date!
```

```
text = "The children were playing happily"
morphological_parser(text)
```

```
Word            POS Tag         Morphological Info
--------------------------------------------------
The             DT              Other
children                NNS             Plural Noun
were            VBD             Verb (Past Tense)
playing         VBG             Verb (Gerund/Present Participle)
happily         RB              Adverb
```